

MMoE and GradNorm: A First-Principles Derivation of Multi-Task Learning

A derivation note that preserves the continuity of thought

Writing Principle

This is not a lecture reverse-engineered from a “standard architecture diagram,” but a note organized in the order “**problem statement** → **build the structure yourself** → **hit a wall / correct** → **converge**.” The MMoE forward pass and the whole GradNorm mechanism (relative training speed, target, L_{grad} , updating only w_k) are **derived step by step** in the text, not memorized. Several “forks” (is a shared expert the same as shared-bottom? soft vs sparse? can w_k be predefined? gradient projection vs gradient norm) are deliberately preserved, because ambiguity comes precisely from these places.

Table of Contents

1. Starting point: what does multi-task learning solve
 2. Architecture evolution: Shared-Bottom → MoE → MMoE
 3. Fork one: is a shared expert the same as shared-bottom?
 4. Fork two: soft (MMoE) vs sparse (LLM MoE) — how far apart is the math?
 5. MMoE forward derivation: $H = GY$
 6. Input layer: shared embedding — where x comes from
 7. Why GradNorm is needed: gradient imbalance in multi-task
 8. Fork three: gradient conflict (projection) vs gradient imbalance (norm) — two roads
 9. GradNorm derivation (1): relative training speed r_k
 10. Fork four: can w_k be predefined? can the scale be scheduled ahead?
 11. GradNorm derivation (2): target and L_{grad}
 12. GradNorm derivation (3): update only w_k , detach the target
 13. The full loop and training procedure
 14. What this derivation actually trained
-

1. Starting point: what does multi-task learning solve

In recommendation/ranking systems, one model often optimizes several objectives at once: CTR (click), CVR (conversion), dwell time, likes...

These are **multiple tasks** sharing the same input features.

Core tension: these tasks are both related (all about “the user’s interest in content”) and conflicting (high-click isn’t necessarily high-conversion). We want: - **Sharing:** let related tasks help each other (knowledge transfer); - **No mutual drag:** conflicting tasks shouldn’t harm each other through sharing (negative transfer).

Multi-task architecture evolution is all about “how much to share, and how.”

2. Architecture evolution: Shared-Bottom → MoE → MMoE

Gen 1: Shared-Bottom

```
x → [one shared MLP bottom] → task_A_tower → y_A
                                ↳ task_B_tower → y_B
```

The bottom is a **single** shared MLP producing **one** representation h ; all tasks get the **same** h , then each goes to its tower. **No gate**. Problem: the bottom is forced to serve all tasks, pleasing none when they conflict (severe negative transfer).

Gen 2: MoE (single-gate) Replace the single bottom with “multiple experts + **one shared gate**.” Experts specialize now, but all tasks share the same gate, still forced to use the same expert combination.

Gen 3: MMoE (multi-gate) Experts shared, but **one gate per task**. Conflicting tasks can, via their own gate, pick different combinations from the shared expert pool, each taking what it needs. The “**multi**” in multi-gate refers to multiple gates (one per task), not multiple experts.

3. Fork one: is a shared expert the same as shared-bottom?

A natural question during derivation: “In MMoE the N experts are the same batch for all tasks — isn’t that just shared-bottom, only with the bottom swapped from one MLP to N experts?”

This intuition captures “the sharing spirit is continuous” (correct), but **equating** the two is wrong. Key difference:

	Shared-Bottom	MMoE shared expert
bottom	1 MLP	N experts
output	1 representation h	N representations $e_1...e_N$
each task gets	the same h (forced identical)	its own weighted combo (can differ)
can it “pick” ?	no	yes, gate decides the mix

- Shared-Bottom’ s “shared” = all tasks **forced to use one output**.
- MMoE’ s “shared expert” = the expert pool is **open to all tasks**, but each task uses **its own gate** to combine into a **different** representation.

Verification (key): if all tasks’ gates output identical weights (say uniform $1/N$), MMoE degenerates to “all tasks get the same combo” $\approx$ shared-bottom. **It’ s precisely the “different gate per task” that distinguishes it from shared-bottom. Remove multi-gate, and MMoE collapses back to shared-bottom.**

Analogy: shared-bottom is a canteen with one big pot everyone eats from; MMoE is a shared dish pool of N dishes, where each person (task) fills their plate by their own taste (gate) — dishes shared, plates different.

Conclusion: what’ s shared is the expert **pool**; what differs is each task’ s **gate mix**. “Shared = pool open,” not “shared = forced identical.” MMoE’ s innovation is the **per-task gate**, not “swapping the bottom to multiple experts” — calling it “shared-bottom with a new bottom” loses the innovation.

4. Fork two: soft (MMoE) vs sparse (LLM MoE) — how far apart is the math?

Another natural judgment: “**Sparse MoE is better, right? Inference only activates top-k, saving compute and latency.**” — In the **LLM setting**, correct (Switch/Mixtral exist for large capacity + saved compute). But see clearly that MMoE and sparse MoE **solve different problems**:

	Sparse MoE (LLM)	MMoE (multi-task)
purpose	expand capacity, save compute	resolve task conflict
#experts	many (64+)	few (4-8)
gate/router	one router per token	one gate per task

	Sparse MoE (LLM)	MMoE (multi-task)
activation	top-k sparse	all, soft weighted

“How far apart is the math?” — barely, only a summation range. The forward pass is identical (N independent experts, gate softmax); the only fork is the combination step:

$$\text{MMoE(soft): } h = \sum_{i=1}^N g_i e_i(x) \quad \text{Sparse: } h = \sum_{i \in \text{TopK}(g)} \tilde{g}_i e_i(x)$$

But this one TopK brings two nontrivial consequences (sparse’ s real cost): 1. **TopK is non-differentiable**: unselected experts get no gradient → expert collapse (a few experts monopolize). 2. **Needs a load-balancing auxiliary loss**: to force the gate to use experts evenly ($L_{aux} = \alpha N \sum_i f_i P_i$). MMoE, fully activated, needs none of this.

Why multi-task defaults to soft MMoE: experts are few (full activation is cheap, saves little latency); multi-task wants soft mixing (top-k hard selection drops soft information); no collapse/load-balancing training headaches.

Terminology: one task monopolizing one expert (task A always uses expert 1) is **not MoE**, it’ s **ensembling**. MoE’ s core is “shared expert pool + gate mixing,” not monopoly.

5. MMoE forward derivation: $H = GY$

Now derive the forward pass from scratch. Let there be K tasks and N experts.

Expert (shared pool): each expert is an MLP:

$$y_i = \sigma(W_i x + b_i), \quad i = 1..N$$

x is $(1, d_x)$, y_i is $(1, d_y)$, so W_i is (d_x, d_y) .

Gate (one per task):

$$g_k(x) = \text{softmax}(W_g^k x + b_g^k), \quad k = 1..K$$

What is the gate’ s output dimension? — locked to N by the operation that follows. This is not a free parameter: the gate must give a weight to each of the N experts, so $g_k(x) \in \mathbb{R}^N$ (W_g^k is (d_x, N)). The gate’ s internal hidden layers are free to design, but the **output must equal N** (to weight N experts). Like a classifier’ s last layer equals the number of classes.

This is best **derived yourself from the matrix-multiplication dimension compatibility**, not memorized: “for the next step $h_k = g_k(x) [y_1 \dots y_N]^T$ to hold, g_k ’s column count must equal the number of experts N .” The operation forces the dimension, not a preset.

Weighted combination (MMoE’ s core operation): task k combines N experts with its own gate weights:

$$h_k = \sum_{i=1}^N g_k(x)_i \cdot y_i = g_k(x) [y_1; \dots; y_N]$$

Matrix form: let G be (K, N) (each row a task’ s gate weights), Y be (N, d_y) (each row an expert output):

$$\boxed{H = GY}, \quad H \text{ is } (K, d_y)$$

Row k of H , h_k , is task k ’ s fused representation. **Each task’ s h_k differs, because each row of G differs — this is the core of multi-gate.**

Tower (task-specific):

$$\hat{y}_k = \text{tower}_k(h_k)$$

The tower’ s structure depends on the task: MLP (most common), RNN/LSTM (sequences), Transformer (attention), CNN — any deep NN. Usually shallow (the heavy representation learning is already done at the expert layer).

6. Input layer: shared embedding — where x comes from

An industrial-grounding note: the x above is not the raw input, but the output of a **shared embedding layer**. Raw features are heterogeneous:

- **sparse features** (user_id, item_id, category) → embedding table → dense vector
- **dense features** (price, duration, statistics) → normalize / bucketize
- **pretrained embeddings** → plugged in directly
- all concat/pooling → unified embedding x , $(1, d_x)$

raw heterogeneous features → [shared embedding layer] → x → N
expert + K gate → $H=GY$ → K tower

Terminology: call this the **shared embedding layer / input layer**, not “shared-bottom” (which specifically means the single MLP bottom that MMoE replaced, see Section 3). “Shared” here means all experts/gates share the same embeddings — a **global sharing even**

more fundamental than the experts — since the sparse embedding table is the bulk of parameters (billions of user/item), and changing it affects all tasks.

7. Why GradNorm is needed: gradient imbalance in multi-task

The most naive total loss:

$$L_{total} = \sum_{k=1}^K w_k L_k$$

Each L_k is task-defined (CE / MSE / ...).

Problem: different tasks have **very different gradient scales** (loss magnitudes differ, e.g. CTR's CE ≈ 0.5 , some regression MSE ≈ 100). If w_k is fixed, **the large-gradient task dominates training**, and small tasks can't learn — optimization is pulled toward the big task, underfitting the rest.

Goal: dynamically adjust w_k so each task's gradient on the shared parameters is **balanced**, letting none dominate. Intuitive benefit: the optimization path isn't yanked into wild oscillation by one large-gradient task; all tasks converge smoothly together.

Key localization: imbalance happens in the **shared part** (experts/embedding/shared layer) — multiple tasks' gradients "fight" there. Task-specific towers each mind their own, no fighting. So what must be balanced is **each task's gradient on the shared parameters W** .

W is usually taken as the **last layer of the shared part** (representative enough, and cheap — computing the norm over all shared parameters is expensive).

8. Fork three: gradient conflict (projection) vs gradient imbalance (norm) — two roads

A very natural brainstorm during derivation: **"Make each task's gradient not affect the others in some direction, so that back-and-forth variation in that direction is 0, ensuring one gradient doesn't affect another."**

This intuition is **sharp**, but it points to a different branch of technique, not GradNorm:

- **What you brainstormed (gradient projection / PCGrad)**: handles gradient **direction conflict** — for two task gradients g_i, g_j , if $g_i \cdot g_j < 0$ (conflict), project g_i onto g_j 's orthogonal complement: $g'_i = g_i - \frac{g_i \cdot g_j}{\|g_j\|^2} g_j$, removing the conflicting component. **This is exactly the precise form of “variation is 0 in that direction.”** Keywords: direction, projection, inner product, orthogonality.
- **GradNorm**: handles gradient **magnitude imbalance** — balances the **norms** of task gradients (by training speed). Keywords: norm, magnitude, training speed.

Two roads solve different problems: one manages direction conflict, one manages magnitude imbalance. The reason the derivation feels twisted is that “map to some direction, variation is 0 in that direction” is the language of projection (direction), while GradNorm is the language of norms (magnitude).

This note derives **GradNorm (norm balancing)**. (PCGrad is a matter for another note.)

9. GradNorm derivation (1): relative training speed r_k

To balance gradient magnitudes, we first need a **cross-task-comparable “learning speed” ruler**.

Step 1: remove magnitude. Different tasks' absolute loss values aren't comparable (magnitude gap). Use the **ratio relative to the start**:

$$\tilde{L}_k(t) = \frac{L_k(t)}{L_k(0)}$$

current loss / initial loss. **Dimensionless**, removing the magnitude gap. $\tilde{L}_k$ small = loss dropped a lot = learning **fast**; large = learning **slow**.

Here you must use the **ratio**, not the current loss L_k itself. Using $L_k / \sum L_j$ directly would be dominated by loss magnitude (large-magnitude tasks always take a big share), unrelated to “learning speed.” You must first divide by the initial loss to remove magnitude.

Step 2: normalize (cross-task comparable).

$$r_k(t) = \frac{\tilde{L}_k(t)}{\frac{1}{K} \sum_j \tilde{L}_j(t)}$$

r_k = task k 's ratio $\div$ the average of all tasks' ratios = **relative training speed**.

- $r_k > 1$: slower than average; $r_k < 1$: faster than average.
- Now $r_A > r_B$ clearly means “A learns slower than B,” undisturbed by loss units — **comparable**.

The $\frac{1}{K}$ in the denominator is optional scaling: drop it (use the sum) and r_k becomes a “share” (summing to 1); keep it and r_k becomes “relative to average” (around 1). **The essence is normalization; $\frac{1}{K}$ only affects the semantics (share vs relative-average), not the relative relationship** — the constant factor is absorbed later.

10. Fork four: can w_k be predefined? can the scale be scheduled ahead?

A recurring idea during derivation: “Give each task’ s gradient norm a scale, $\|w_k \nabla_W L_k\| \leq \text{scale}$, and this scale can be predefined — e.g. 0.1 for the first 100 steps, 0.05 for steps 100–1000, and with 10 tasks split it evenly as $\text{scale}/10$.”

This scheme **runs and is self-consistent**, but it is **not GradNorm**, differing in two essential ways (which are GradNorm’ s two innovations):

Difference 1: even split vs uneven split by training speed. “ $\text{scale}/10$ even split” assumes “all tasks should have the same gradient norm.” But GradNorm’ s core is precisely **uneven splitting** — the slow-learning task should get a **larger** target (to push it), the fast one a smaller one. Even splitting erases the inter-task learning-speed difference, and handling that difference is exactly why GradNorm exists.

Difference 2: preset schedule (open-loop) vs real-time-computed baseline (closed-loop). A “scale scheduled by step count” is **open-loop**: it ignores each task’ s real state, follows a preset timetable, and requires hand-tuning that timetable. GradNorm uses “**the current average of all tasks’ gradient norms**” as the baseline, **closed-loop and adaptive**, no preset scheduling.

The key shift in thinking: don’ t think “give the scale ahead of time,” but ask — “**at this moment, how large should a task’ s gradient norm be to be ‘in line’ ?** The answer: **in line with the current average of all tasks.**” This average is **computed in real time, not preset**.

Another correction: w_k **cannot be predefined and fixed**. w_k is exactly what GradNorm **learns dynamically** — if predefined, it degenerates into ordinary fixed weighting, and GradNorm isn't needed. w_k is optimized by a loss (Sections 11–12), not an analytic formula.

11. GradNorm derivation (2): target and L_{grad}

With the ruler r_k in hand, now build the target and the optimization objective.

Each task's gradient norm (the quantity to balance):

$$G_W^{(k)} = \|\nabla_W(w_k L_k)\|_2 = \|w_k \nabla_W L_k\|_2$$

The norm of task k 's **weighted** loss gradient w.r.t. the shared parameters W . w_k is multiplied in.

Target gradient norm:

$$\text{target}_k = \bar{G}_W \cdot r_k^\alpha, \quad \bar{G}_W = \frac{1}{K} \sum_k G_W^{(k)}$$

- $\bar{G}_W$ (current average norm) = **baseline magnitude** (computed in real time, not preset, providing the “absolute size”). - r_k^α = **relative adjustment**: $r_k > 1$ (slow) $\rightarrow$ target **large** (push it); $r_k < 1$ (fast) $\rightarrow$ target **small**. α controls the push strength.

Why must the target be multiplied by $\bar{G}_W$ and not simply equal r_k ? Because r_k is dimensionless (magnitude ≈ 1), while gradient norms have a real magnitude. Setting the norm $= r_k$ directly would wrongly pin the norm near 1. Multiplying by $\bar{G}_W$ provides the magnitude; r_k handles only the relative adjustment. And $\bar{G}_W = \frac{1}{K} \sum \|w_k \nabla_W L_k\|$ is itself “the average of current weighted gradient norms” — carrying its own magnitude.

L_{grad} : force the actual norm toward the target. Use squared distance (MSE) or L1:

$$L_{grad} = \sum_k (G_W^{(k)} - \text{target}_k)^2$$

(The original paper uses L1 $\sum_k |G_W^{(k)} - \text{target}_k|$; MSE also works, same direction.)

It is an independent auxiliary objective, not added to the original cost. This is GradNorm's key structural feature: - $L_{total} = \sum_k w_k L_k \rightarrow$ updates network parameters θ ; - $L_{grad} \rightarrow$ updates only the weights w_k , not touching the network.

It is not a “regularization term added to the main loss” (which would be differentiated together with the network parameters), but an independent objective optimizing w_k .

12. GradNorm derivation (3): update only w_k , detach the target

How is w_k updated? — L_{grad} is differentiated w.r.t. w_k , gradient descent:

$$w_k \leftarrow w_k - \eta \frac{\partial L_{grad}}{\partial w_k}$$

- r_k large (slow) $\rightarrow$ target large $\rightarrow$ actual $G_W^{(k)} <$ target $\rightarrow L_{grad}$ pushes w_k larger $\rightarrow$ the slow task's weight increases.
- This matches “ w_k positively correlated with r_k ” : **the slow gets a large weight**. (Note it's not “the direct reciprocal” — setting $w_k = L_0/L_k$ directly would reverse the direction; w_k is optimized by L_{grad} , not an analytic formula.)

Two key engineering details (most error-prone):

1. **When computing $G_W^{(k)}$, keep the graph differentiable w.r.t. w_k .** That is, in $G_W^{(k)} = \|w_k \nabla_W L_k\|$, w_k is inside the norm; use `autograd.grad(..., create_graph=True)` so L_{grad} can be differentiated w.r.t. w_k .
 2. **The target must be detached (treated as a constant).** The target contains $\bar{G}_W$ (which contains gradient norms), but when computing $\partial L_{grad} / \partial w_k$, **treat the target as constant**, letting only the actual term $G_W^{(k)}$ be differentiated w.r.t. w_k . Otherwise the gradients go haywire and training is unstable.
 3. **Renormalize:** after each step, normalize w so that $\sum_k w_k = K$ (preventing all w_k from drifting away together, equivalent to changing the global learning rate).
-

13. The full loop and training procedure

Each step:

1. **Forward:** $H = GY$, each task $\hat{y}_k = \text{tower}_k(h_k)$, compute L_k .
2. **Main update:** $L_{total} = \sum_k w_k L_k$, `backward` to update network parameters θ (w_k as constant).

3. **Compute training speed:** $\tilde{L}_k = L_k(t)/L_k(0)$, $r_k = \tilde{L}_k/\bar{\tilde{L}}$.
4. **Compute gradient norms:** $G_W^{(k)} = \|w_k \nabla_W L_k\|$ (w.r.t. shared W , `create_graph=True`).
5. **Target:** $\text{target}_k = \bar{G}_W \cdot r_k^\alpha$ (**detach**).
6. **GradNorm update:** $L_{grad} = \sum_k (G_W^{(k)} - \text{target}_k)^2$, `backward` to update only w_k .
7. **Renormalize:** $w \leftarrow w \cdot K / \sum_k w_k$.

Effect: under fixed weights the big task dominates and the small task can't learn; under GradNorm w auto-adjusts (big-task weight $\downarrow$, small-task weight $\uparrow$), gradients balance, and tasks converge together.

14. What this derivation actually trained

1. **MMoE's innovation is multi-gate, not multiple experts:** a shared expert pool + a different gate mix per task $\rightarrow$ mitigates negative transfer. Remove multi-gate and it collapses to shared-bottom.
 2. **soft vs sparse forward differs only in the summation range** (all vs top-k), but non-differentiable TopK brings the engineering complexity of collapse/load-balancing — that's the essential difference.
 3. **Dimensions are locked by the operation, not preset:** the gate's output = N , forced by the matrix compatibility of $H = GY$.
 4. **GradNorm balances gradient norms (magnitude), not loss magnitude;** it is orthogonal to gradient projection (direction conflict) — two roads.
 5. **Training speed must use the ratio $L_k(t)/L_k(0)$ to remove magnitude,** then normalize to be comparable.
 6. **Don't predefine the scale / don't predefine w_k :** GradNorm's essence is “compute the baseline in real time (current average gradient norm) + split unevenly by training speed,” not “preset schedule + even split.”
 7. **Two losses each mind their own parameters:** L_{total} updates the network, L_{grad} updates only w_k ; the target must be detached.
-

Compressed to one sentence: MMoE = shared expert pool + one gate per task (soft weighting $H = GY$) + task towers; GradNorm = using “current average gradient norm $\times$ relative training speed r_k ” to set the target, building an independent L_{grad} that optimizes only the task weights w_k (target detached), dynamically balancing each task's gradient norm on the shared parameters — uneven, not preset.